DATA ENGINEERING & CLOUD · LESSON 10 OF 15

Data Quality Dimensions, Observability, Catalogues, Lineage and Classification

Define data quality with measurable rules across completeness, validity, uniqueness, consistency, timeliness and accuracy. Build a quality check suite that records results, monitor pipelines for freshness, volume and schema changes, and use catalogues, lineage and classification to find and protect data.

Gap-fill lesson | ✦ 2026 | Intermediate | ⏱ 120 min

By the end of this lesson you can

- Define data quality dimensions and turn each into a measurable rule
- Build a quality check suite that records pass or fail and counts
- Monitor freshness, volume and schema change as observability signals
- Use a catalogue, lineage and classification to find and protect data

Datasets used

`superstore_sales.csv` `customers.csv`

Quality as measurable rules

"The data is good" means nothing until it is defined. Data quality is measured along dimensions, and each dimension becomes a rule that a query can test.

Dimension	Question	Example rule
Completeness	Are required values present?	`customer_id` is never null
Validity	Do values follow the allowed format or range?	`discount` between 0 and 1
Uniqueness	Does each entity appear once?	`row_id` is unique
Consistency	Do related values agree?	every `customer_id` in sales exists in customers
Timeliness	Is the data recent enough?	the newest order is at most one day old
Accuracy	Does the data match the real world or a trusted source?	daily totals match the finance ledger

Accuracy is the hardest dimension, because it needs a trusted reference. The others can be checked from the data itself.

CASE STUDY **The report that was complete and wrong**

A dashboard showed no missing values and no duplicates, and still reported 3% less revenue than finance. The rule set had no accuracy check. A reconciliation against the ledger found that one region's file had been loaded with an old exchange rate. Completeness and validity were fine; accuracy was not measured.

A quality check suite

Each check is a query that counts failing rows. Zero means pass. Store every result, so that trends are visible and a single failure can be investigated.

Python

```
import pandas as pd
import duckdb

con = duckdb.connect()
sales = pd.read_csv("superstore_sales.csv", parse_dates=["order_date"])
customers = pd.read_csv("customers.csv")
con.register("sales", sales)
con.register("customers", customers)

checks = {
    "completeness: customer_id is null":
        "SELECT COUNT(*) FROM sales WHERE customer_id IS NULL",
    "uniqueness: duplicate row_id":
        "SELECT COUNT(*) - COUNT(DISTINCT row_id) FROM sales",
    "validity: discount outside 0..1":
        "SELECT COUNT(*) FROM sales WHERE discount < 0 OR discount > 1",
    "validity: non-positive quantity":
        "SELECT COUNT(*) FROM sales WHERE quantity ⩽ 0",
    "consistency: sales customer not in customers":
        "SELECT COUNT(*) FROM sales s LEFT JOIN customers c USING (customer_id) WHERE
c.customer_id IS NULL",
}

results = []
for name, sql in checks.items():
    failing = con.execute(sql).fetchone()[0]
    results.append({"check": name, "failing_rows": failing, "status": "PASS" if failing ==
0 else "FAIL"})

report = pd.DataFrame(results)
print(report.to_string(index=False))
```

OUTPUT

```
                                    check  failing_rows status
           completeness: customer_id is null             0   PASS
                uniqueness: duplicate row_id             0   PASS
             validity: discount outside 0..1             0   PASS
             validity: non-positive quantity             0   PASS
consistency: sales customer not in customers             0   PASS
```

The consistency check is the one that most often fails in practice. A sales row whose customer was deleted from the customer table disappears from a join and quietly understates totals.

::::tip Make the failure useful A check that says "FAIL" is not enough. Also store the failing rows (or a sample), the check name, the date and the threshold. Then the person on call can see what broke without rerunning the job. ::::

Thresholds and severity

Not every failure stops the pipeline. Decide severity in advance:

Severity	Example	Action
Blocking	Null primary key, duplicate keys	Stop the pipeline; do not publish
Warning	0.5% of discounts out of range	Publish with a flag; alert the owner
Informational	Row count up 12% week on week	Record for trend review

Thresholds should come from the business. A 0.5% tolerance may be acceptable for marketing data and unacceptable for invoices.

Observability: knowing when something changes

Quality rules check the data you expect. **Observability** watches for the things you did not expect. Three signals cover most cases:

1. **Freshness.** When did the newest record arrive? Alert if it is older than the agreed limit.
2. **Volume.** How many rows arrived? Alert on sharp drops or spikes against a recent average.
3. **Schema.** Did a column appear, disappear or change type? Alert before the change breaks a downstream model.

Python

```python
import pandas as pd

history = pd.Series([1000, 1040, 980, 1010, 1025, 990, 1005])   # daily row counts, last week
today = 430
mean, std = history.mean(), history.std()
z = (today - mean) / std
print(f"today={today}, mean={mean:.0f}, z={z:.1f}")
print("ALERT: volume drop" if z < -3 else "volume within normal range")
```

OUTPUT

```
today=430, mean=1007, z=-28.3
ALERT: volume drop
```

A simple statistical rule such as "more than three standard deviations from the recent mean" catches most real incidents without tuning. Use a seasonal baseline for data with weekly patterns.

Catalogues, lineage and classification

A **data catalogue** is the inventory of datasets: what each one contains, who owns it, how fresh it is, and how to get access. A catalogue entry needs a description, an owner, a refresh schedule and a link to its quality results.

Lineage records where a dataset came from and what depends on it (see the dbt lesson for model-level lineage). It answers two questions: "why is this number wrong?" (trace upstream) and "who will be affected if I change this?" (trace downstream).

Classification labels columns by sensitivity: public, internal, confidential, personal. Classification drives access rules and masking. Classify at ingestion, store the label in the catalogue, and apply the masking rule in the layer that consumers read.

Column	Classification	Rule
order_id	Internal	Available to analysts
customer_name	Personal	Masked in silver and gold, restricted in bronze
sales	Internal	Available to analysts and finance
card number	Personal, high risk	Never loaded in raw form (see the ETL lesson)

INTERVIEW CORNER

How would you know a dashboard number is wrong before a business user does?

Combine rules and signals: completeness and consistency checks on the source, freshness and volume monitoring on each load, reconciliation with a trusted total where one exists, and lineage to find the upstream table if something drifts. Alert the owner when a check fails, and show a quality status next to the dashboard.

::::key What to remember - Turn each quality dimension into a measurable rule, and record every result. - Accuracy needs a trusted reference; the other dimensions can be checked from the data. - Decide severity and thresholds with the business in advance. - Observe freshness, volume and schema to catch what you did not expect. - Catalogues, lineage and classification make data findable and protected. ::::

Quick quiz

5 questions · tap an answer to check it

1. Which dimension asks whether each entity appears once?

A Validity

B Timeliness

C Uniqueness

D Accuracy

Answer: C. Uniqueness checks for duplicates of an entity's key.

2. A check returns 0 failing rows. What does that mean?

A The check did not run

B The rule passed for every row checked

C The data is accurate

D The table is empty

Answer: B. Zero failing rows is a pass, though it says nothing about accuracy.

3. Why do consistency checks often reveal understated totals?

A They add rows

B A sales row with no matching customer drops out of an inner join, so totals fall silently

C They remove duplicates

D They change currency

Answer: B. Missing parent rows make joins lose data.

4. Which observability signal would catch a source that stopped sending files?

A Schema

B Freshness

C Accuracy

D Uniqueness

Answer: B. Freshness measures how recent the newest data is.

5. Why classify columns by sensitivity?

A To change their names

B To decide who may see them and where masking must be applied

C To speed up queries

D To remove the need for a catalogue

Answer: B. Classification drives access and masking rules.

EXERCISE **Build a suite and a severity table**

For the customers and sales files, write five quality checks (one per dimension, where the dimension applies), assign each a severity, and run them. Report which fail, what you would do about each, and one accuracy check you could not run and what reference you would need.

Show solution (worked answer)

Hints: the consistency check uses a LEFT JOIN with a NULL test. For accuracy, name a reference such as a finance ledger total by month. Put the severity in a column of the report table, so the decision is visible.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.